Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI5CC · CLOUD COMPUTING · WEEK 3

Cloud Infrastructure and Compute

The layers of the cloud, virtualization, containers, and Amazon EC2.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Recap

LAST SESSION

Service models

IaaS, PaaS and SaaS: how much of the stack the vendor manages.

LAST SESSION

Deployment models

Public, private, hybrid and multi-cloud: who owns the cloud and who shares it.

Learning objectives

1. Describe the layers of the cloud computing stack.
2. Explain virtualization concepts: virtual machines, hypervisors and containers.
3. Describe EC2 fundamentals, instance types and pricing.
4. Explain AMIs and snapshots.
5. Launch and connect to an EC2 instance (Lab 1).

THIS WEEK

Topic 3

Cloud infrastructure, compute services (EC2) and virtualization. Assessed by class participation, a quiz and a lab report (CLO1, CLO2).

PART 1

Common cloud services

The layers of the cloud computing infrastructure stack.

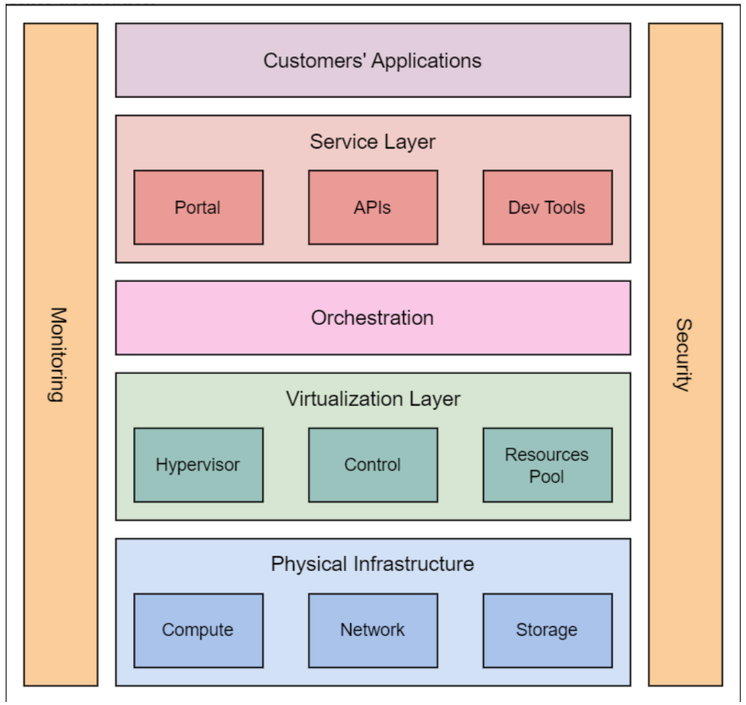
Week 3 · 04

The layers of the cloud

A high-level view of how various components are stacked to provide the cloud services we all use, on any given platform.

It does not represent the architecture of any particular cloud platform.

Monitoring and security run alongside every layer.



The layers and components of the cloud architecture

Five layers, bottom to top

01

Physical infrastructure

Physical servers, network equipment and storage devices, in datacenters that vendors operate worldwide.

02

Virtualization

Software that virtualizes the physical hardware. Its key component is the hypervisor.

03

Orchestration

Distributed systems that find servers to fulfil requests, replicate resources and reroute around failures.

04

Service

Portals, web and mobile applications, APIs, CLIs, SDKs and libraries to use the platform.

05

Customer application

Where customers use those services to create their business applications.

SO WHAT?

Why it matters

Customers are shielded from this complexity, and interact with the cloud through simple APIs.

Physical infrastructure layer

Components

Physical servers, network equipment and storage devices.

Datacenters

House racks of servers containing CPUs, memory and hard drives.

Network connection

All components are interconnected via a network within the datacenter.

Global management

Cloud vendors operate datacenters worldwide to cater to a global audience.

The physical layer on AWS

01

Region

A geographic area, such as Singapore, that holds multiple Availability Zones. We choose the Region our resources run in.

02

Availability Zone

One or more datacenters with their own power, cooling and networking, isolated from the other zones in the Region.

03

Edge location

A site close to users that caches content, used by content delivery services such as Amazon CloudFront.

Spreading an application across Availability Zones keeps it running when one datacenter fails.

Virtualization and orchestration layers

Virtualization layer

Operations on cloud platforms are performed through virtualization, not directly on bare metal hardware. It is a core aspect of cloud computing.

Orchestration layer

Orchestrators locate servers to fulfil service requests. Resources are replicated across servers and regions, and requests are rerouted to available servers when a node fails.

Scalability, resilience and robustness come from distributed systems layered over virtualization.

PART 2

Compute and virtualization

Virtual machines, hypervisors, containers and scaling.

Week 3 · 10

What is compute?

Processing power, memory, storage and network resources together are called *compute*.

What applications need

Software applications need all four kinds of resource to run.

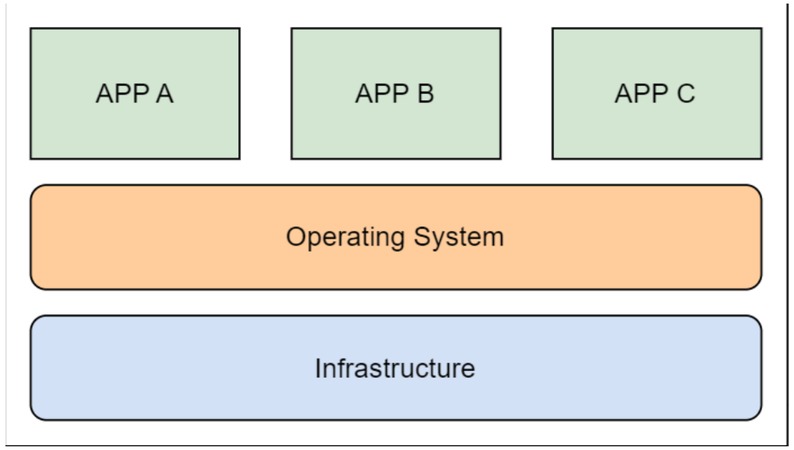
Fundamental

Computing services are fundamental, and used in all aspects of cloud activities.

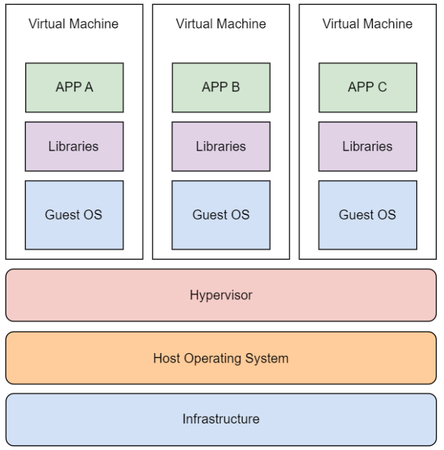
A variety of services

Cloud platforms offer a range of computing services suited to different scenarios.

Virtual machine (VM)



Traditional: one operating system



Virtualized: a guest OS in each VM

A virtual machine is essentially a software-based emulation of a physical computer.

Key points about VMs

Software emulation

VMs mimic the functionality of a physical computer, including a CPU, memory, storage and network capabilities.

Digital representation

They can be thought of as digital versions of physical machines, fully managed through software.

Flexibility

Each VM runs its own operating system and applications, independently of the host machine's OS.

We can run multiple virtual machines on the same machine. This capability is enabled by the hypervisor software.

Hypervisors: two types

IN THE DATACENTER

Type 1: bare metal

Runs directly on the physical hardware, with no host operating system underneath. Used in datacenters and by cloud providers. Examples: VMware ESXi, Microsoft Hyper-V, KVM and Xen.

ON YOUR LAPTOP

Type 2: hosted

Runs as an application on top of a host operating system. Used on laptops and desktops. Examples: Oracle VirtualBox and VMware Workstation.

The hypervisor shares one machine's CPU, memory and disk among many VMs, and keeps them isolated from each other.

VMs in cloud computing

Building blocks

Fundamental elements for obtaining computing power from the cloud.

Easy to create

Spinning up VMs is straightforward, as many as needed.

Scalable and flexible

Create, scale up or scale down VMs based on demand.

Resources on tap

Vendors provide almost limitless resources, with no hardware to manage.

Customizable

Configure CPU, GPU, RAM and disk space to suit each application.

On every platform

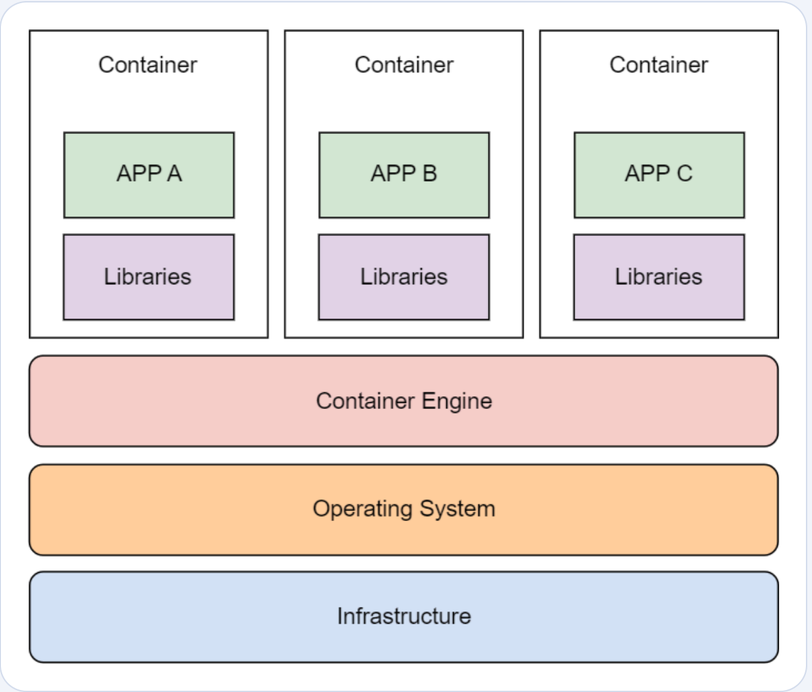
Azure Virtual Machines, Amazon EC2 and Google Compute Engine.

Containers

A software package with *everything* an application needs to run.

Code, runtime, configurations and dependencies. It is portable: easily moved from one computing environment to another.

It shares the host's operating system but keeps the application isolated, so it is lighter than a VM, which needs a full operating system for each instance.



Why containers took off

Since 2013

Docker’s introduction in 2013 significantly increased the usage of containers in cloud computing.

Container images

Blueprints for creating containers, ensuring consistency across different instances.

Agility

Quick and confident development, deployment and management of applications.

Microservices

Their small size and scalability make containers ideal for a microservices architecture.

Virtual machines vs containers

	Virtual machine	Container
Virtualizes	The hardware	The operating system
Each instance contains	Application, libraries and a full guest OS	Application and libraries only
Runs on	A hypervisor	A container engine
Size and start-up	Heavier, slower to start	Lightweight, starts quickly
Isolation	Strong: a separate OS each	Process-level: one shared OS

Orchestrating containers

As the use of containers grows, orchestrating them becomes essential to manage their operations and ensure security and reliability. Kubernetes is a leading orchestration engine, widely adopted in the industry.

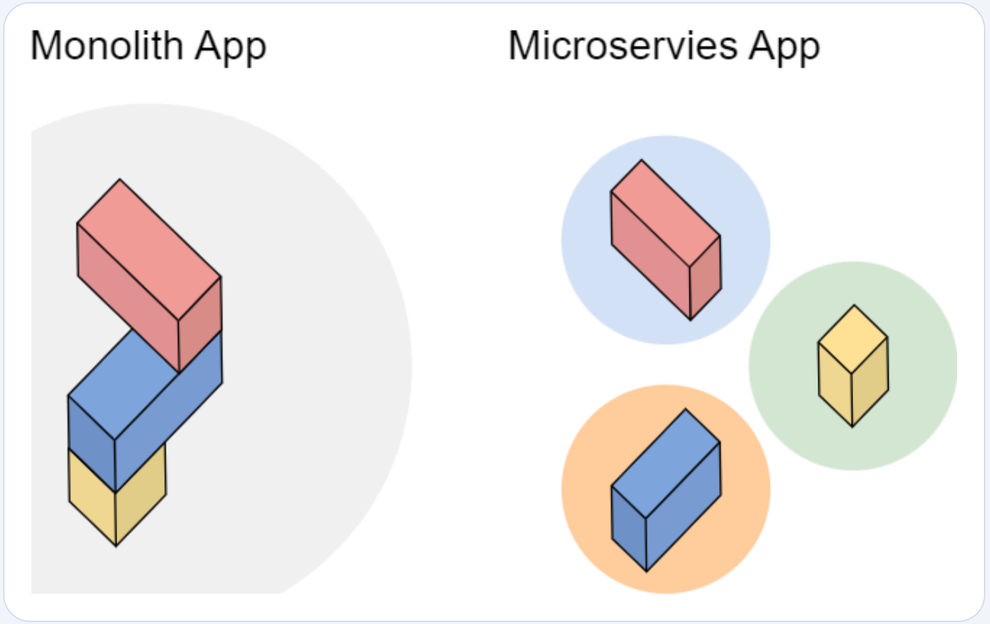
Cloud provider	Managed Kubernetes service
Microsoft Azure	Azure Kubernetes Service (AKS)
AWS	Amazon Elastic Kubernetes Service (EKS)
Google Cloud	Google Kubernetes Engine (GKE)

Microservices

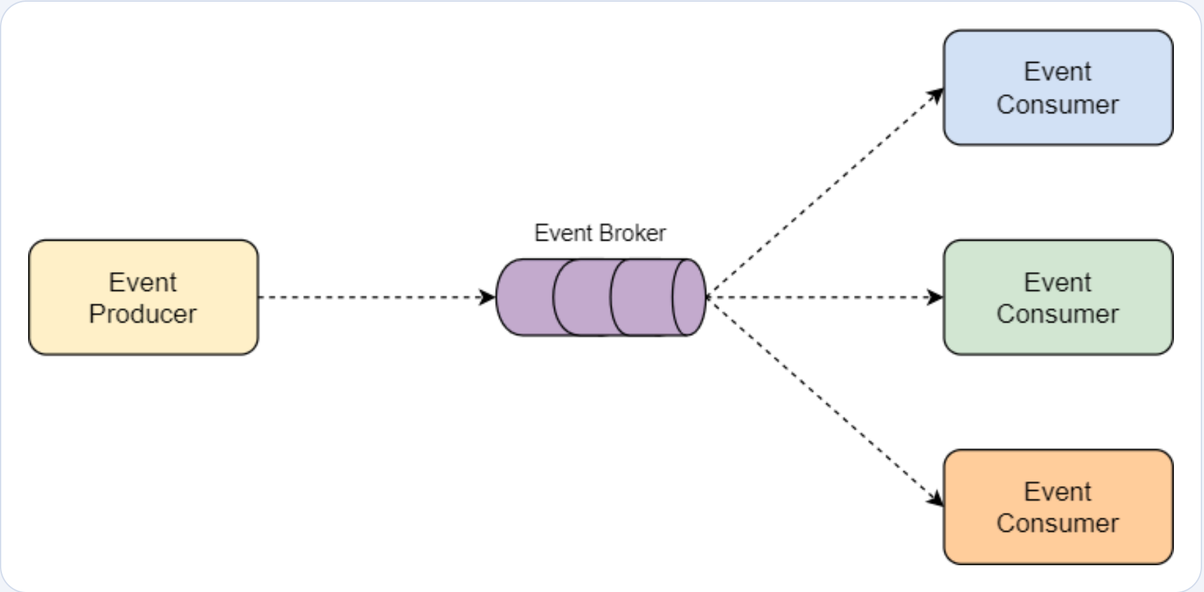
Applications built as *small, independent services* that communicate via APIs.

Each service is loosely coupled, owned by a small autonomous team, and can be deployed and scaled independently, in its own technology stack.

Unlike a monolithic application, where all components are interconnected and interdependent. We return to this in a later session.



Event-driven architecture



Events, signals that a state has changed, are the means of communication between decoupled services: producers, a broker and consumers.

Example: a ride-sharing app

Event producers

Riders and drivers. A ride request is an event. A driver accepting it is another.

Event broker

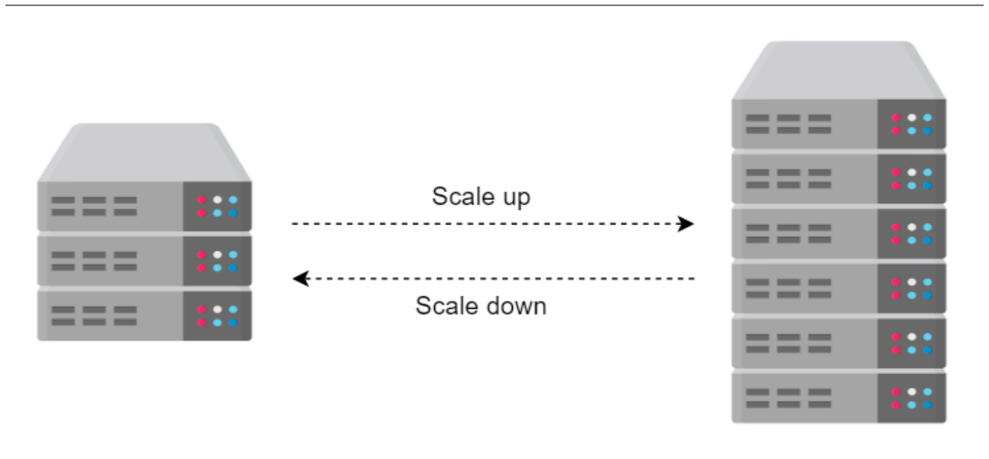
The app's backend receives ride requests and forwards them to available drivers.

Event consumers

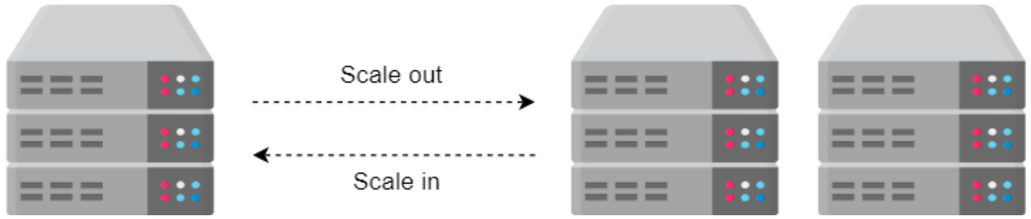
Drivers receive the requests and respond, which triggers further events such as status and location updates.

Nobody polls the server for changes. Updates arrive in real time as events, which reduces server load and cost.

Vertical scaling vs horizontal scaling



Vertical: a more powerful machine



Horizontal: more machines of the same size

Scaling is adjusting resources to meet the application's needs. Horizontal scaling works best for stateless applications.

Scaling up or scaling out?

ONE BIGGER MACHINE

Vertical scaling (scaling up)

Enhancing a single server by adding more RAM, disk space or CPU power. The same machine becomes more powerful. Cloud platforms make it easy to scale up or down with the workload.

MORE MACHINES

Horizontal scaling (scaling out)

Adding more VMs of the same size, and removing them as demand falls. It scales beyond what one machine can offer, and cloud platforms automate it, often with just a click.

Why use compute in the cloud?



Flexibility and control

Lift and shift existing setups onto VMs for full control. Modernize by rewriting applications as containers. Go serverless for event-driven, cloud-native applications.



Cost and performance

A range of options, from VMs to serverless. With serverless models, businesses pay only for execution.



Security

Cloud computing often provides better security than traditional models. Who secures what follows the shared responsibility model.

PART 3

Amazon EC2

Amazon Elastic Compute Cloud: virtual machines on AWS, rented by the second.

Week 3 · 26

EC2 fundamentals

Amazon EC2 provides virtual machines, called *instances*, on demand.

It is IaaS

AWS runs the hardware and the hypervisor. We choose and manage the operating system and everything above it.

Resizable

Start with a small instance, and change its size or add more instances as the workload changes.

Pay for use

We pay while an instance is running, and stop paying for compute when it is stopped or terminated.

What we choose when launching an instance

01

AMI

The template: operating system and preinstalled software.

02

Instance type

How much CPU, memory and network capacity.

03

Key pair

The credentials we use to log in.

04

Network

Which VPC and subnet, and whether it gets a public IP address.

05

Security group

The firewall rules: which traffic may reach the instance.

06

Storage

The size and type of its disks (Amazon EBS volumes).

Instance types

Family	Letters	Good for
General purpose	T, M	Web servers, small databases, development and test
Compute optimized	C	Batch processing, high-performance computing
Memory optimized	R, X	In-memory databases and caches
Storage optimized	I, D	Workloads that read and write large local datasets
Accelerated computing	P, G	Machine learning and graphics, using GPUs

Choose the family from what the workload needs most: CPU, memory, disk or GPU.

Reading an instance type name

t

Family

General purpose, in this case the burstable T family.

3

Generation

A higher number is a newer generation of hardware.

micro

Size

From nano and micro up to large, xlarge and beyond. Each step up roughly doubles the capacity.

So t3.micro is a small, general-purpose instance of the third generation.

EC2 pricing

Option	How it works	Best for
On-Demand	Pay by the second or hour, with no commitment	Short-term, unpredictable workloads, and learning
Savings Plans and Reserved Instances	Commit to 1 or 3 years for a lower price	Steady workloads that run all the time
Spot Instances	Use spare AWS capacity at a large discount. AWS can reclaim it at short notice	Batch jobs that can withstand interruptions
Dedicated Hosts	A physical server for your use only	Licensing or compliance requirements

AMIs and snapshots

A TEMPLATE

Amazon Machine Image (AMI)

A template for launching instances: an operating system, plus any software and configuration. Use one provided by AWS, or create your own from a configured instance to launch identical copies.

A BACKUP

EBS snapshot

A point-in-time backup of an instance's disk (an Amazon EBS volume). Snapshots are incremental: after the first, only the blocks that changed are saved.

An AMI of your own is built from snapshots of the instance's volumes, plus launch settings.

EC2 instance lifecycle

01

Pending

The instance is being launched.

02

Running

Ready to use. Compute is billed.

03

Stopped

Shut down, disk kept. Can be started again.

04

Terminated

Deleted for good.

A stopped instance costs nothing for compute, but its EBS storage is still charged. Terminate what you no longer need.

Connecting to an instance



Key pair

AWS keeps the public key and we download the private key. It proves who we are when we log in.



Security group

The instance’s firewall. To use SSH, allow inbound TCP port 22, ideally from your own IP address only.



SSH

From a terminal, connect to the instance’s public IP address with the private key and the AMI’s default user name.

Lab 1: AWS account and EC2

1. Set up your AWS Academy Learner Lab account.
2. Launch an EC2 instance.
3. Configure its security group.
4. Connect to it via SSH.
5. Do the basic configuration, then clean up.

REMEMBER

Lab report

Document the AWS resources you create, and make sure they are cleaned up after the lab. Watch your usage against the 50 USD quota.

QUIZ

Post-knowledge quiz

The cloud stack, virtualization and containers, and Amazon EC2.

Week 3 · 36

Next session

01

Cloud scaling

02

Load balancing

03

Lab 2